

Gaitform — Project Context & Handoff Document

This document is a complete briefing for any AI assistant picking up this project. Read this in full before responding to any request related to "Gaitform," the orthotic prototype, or the associated patent work. It contains the original problem, the target architecture, the current verified state, known unresolved issues, and the working method that should be carried forward.

1. The original idea and the problem being solved

The goal is a patent and working prototype for a system that converts **frontal-view smartphone gait video directly into a parametric, 3D-printable custom orthotic**. This bridges two things that currently exist separately:

- **Markerless gait analysis from smartphone video** (the closest real-world reference is VisionMD-Gait, an open-source research system published in early 2026, validated against wearable sensors with under 10% mean absolute error on cadence, gait speed, step duration, etc.)
- **Parametric/procedural orthotic CAD tools**, which today only accept static foot shape or pressure-plate data (e.g. Phits, the closest commercial competitor — their input is a static pressure plate, not dynamic video)

The core novelty claim: no existing system takes *dynamic gait video* as the direct input driving *orthotic geometry generation*. Static pressure-plate or foot-scan systems capture shape at rest; this system captures how the foot actually behaves during walking and uses that to drive design parameters (arch height, heel cup depth, post angles, rigidity zoning).

The long-term moat, separate from the algorithm itself: a paired dataset of gait recordings → orthotic parameters → clinical outcomes. The algorithm is patentable; the dataset is the durable competitive advantage. This is why the dataset logger and `clinician_outcome_notes` field were treated as first-class requirements from the start, not an afterthought.

Explicit non-goal: this is a research/demo prototype for patent reduction-to-practice, not a certified medical device. Every output must be framed as decision-support for a clinician, never a diagnosis or auto-dispensed prescription. This framing must never be diluted, regardless of how confident any output looks.

2. Target architecture (as originally specified)

[smartphone video]

- Phase 1: GaitExtractor (MediaPipe primary, VisionMD-Gait adapter alternative)
- Phase 2: Gait Metrics Engine (cadence, stride length, step width, stance/swing %, pronation angle, symmetry)
- Phase 3: Orthotic Parameter Mapper (literature-backed rules, clamped to safe ranges, flags for clinician review)
- Phase 4: Parametric CAD Generator (watertight STL)
- Phase 5: Rigidity Zoning (per-zone rigidity → print_instructions.json + heatmap)
- Phase 6: Dataset Logger (SQLite, append-only, clinician_outcome_notes nullable field)
- Phase 7: Demo UI (Streamlit, "Full pipeline trace mode")
- Phase 8: Polish & packaging (README, mermaid diagram, license list, sample data, v0.1.0 tag)

Repo structure: `gaitform/{extraction,metrics,mapping,cad,logging,ui}/`, `tests/`, `data/`, `sample_data/`.

Tech stack: MediaPipe PoseLandmarker (pose extraction), scipy (signal processing — `find_peaks` on detrended heel/ankle y-trajectory for heel-strike/toe-off detection), trimesh + numpy procedural mesh generation (CadQuery was considered and rejected for being too heavy/conda-dependent), SQLite (dataset logging), Streamlit (UI).

3. The data schema (target)

json

```

{
  "session_id": "uuid",
  "created_at": "iso8601",
  "video_metadata": { "filename": "...", "fps": 30, "duration_s": 8.2, "view": "fronta
  "raw_keypoints_path": "...",
  "gait_metrics": {
    "cadence_steps_per_min", "stride_length_L_m", "stride_length_R_m",
    "step_width_m", "stance_time_pct", "swing_time_pct",
    "pronation_angle_L_deg", "pronation_angle_R_deg", "limb_symmetry_index",
    "foot_length_mm" // added later, see section 5
  },
  "orthotic_parameters": {
    "arch_height_mm", "heel_cup_depth_mm", "medial_post_deg", "lateral_post_deg",
    "met_pad_position_mm", "rigidity_zones": [{"zone", "target"}], "flags_for_review":
  },
  "stl_path": "...",
  "print_instructions_path": "...",
  "clinician_outcome_notes": null // intentionally null at generation; filled retroac
}

```

`clinician_outcome_notes` is the field that turns the log into a training set. It has never been populated with real data — the plumbing exists, no real clinician/patient outcome has been recorded yet.

4. Process notes / IP considerations (important context, not just engineering)

- **Antigravity's default data-sharing terms were flagged as a real concern.** On a personal (non-enterprise/non-GCP) account, Google's terms state that interactions are by default used to evaluate/develop/improve Google products, and staff may access them. This is a third-party disclosure risk for unfilled patent material. The decision of whether to strip "patent" framing out of prompts sent to Antigravity, turn off telemetry, or use an enterprise seat was raised multiple times and **was never definitively resolved** — the README still says "Research Prototype for Patent Demonstration" as of the last review. Revisit this before further public/external sharing.
- **A YouTube-ripped commercial walk-cycle reference video was found in the repo** (filename signature: "YTDown..."), used as test data at one point. This was a real problem (likely-copyrighted stock footage in a patent-adjacent deliverable, plus it was split-screen with two people in frame, undermining test validity). It was identified and replaced with the actual VisionMD-Gait sample video: <https://github.com/mea->

`lab/GaitValidation/blob/main/DEMO/demo.mp4`) — a genuine single-subject, portrait-orientation smartphone recording from the real validation study (confirmed by visible mocap lab setup, sensor straps, floor markers in the frame). **Use only this video, or freshly recorded real smartphone footage, going forward. Never reintroduce downloaded stock/reference footage.**

5. Verified-working state (confirmed by direct, independent code execution — not by trusting any pasted report)

The working method established in this project: **never trust a status report, a screenshot, or pasted terminal text at face value.** Every major claim of "fixed" or "working" in this project's history has needed independent verification, and several confidently-stated claims turned out to be false, stale, or fabricated under direct testing. The pattern that recurred repeatedly: a report says X is fixed; direct execution of the actual code shows X is not fixed, or the "proof" came from a side-script/manual DB insert rather than the real application path. **Any future AI assistant working on this project should adopt the same standard: when given source files, run them directly in a sandbox rather than evaluating claims about them.**

As of the most recent zip-based review (3 core files only: `analyzer.py`, `rules.py`, `generator.py`, plus `demo.mp4` — NOT the full repo), the following were independently confirmed by actually running the code against real, freshly-extracted pose data from `demo.mp4`:

- **Cadence/stride detrending bug — confirmed fixed.** `analyzer.py` correctly applies `scipy.signal.detrend()` to the heel y-trajectory before `find_peaks`, removing the camera-approach perspective drift that was previously causing silent `cadence: 0.0` output. Independently re-extracted real keypoints and got `cadence_steps_per_min: 80.52` (a separate, later self-reported run claimed `93.93...` for reasons explained — see section 6).
- **Orthotic rules engine — confirmed genuinely dynamic, not hardcoded.** Ran `rules.py` with two different pronation inputs (normal $\sim 6.3^\circ$, extreme 15.5°) and got correctly different outputs: `heel_cup_depth_mm` 10.0 $\rightarrow$ 15.0, `midfoot` rigidity medium $\rightarrow$ firm, flags populated correctly on the extreme case. This resolved a long-standing suspicion (across ~ 4 prior runs, four output params were bit-identical regardless of input) — the suspicion was valid to raise, but the current code does not have this problem.
- **CAD heel cup depth bug — confirmed fixed.** Previously, `heel_cup_depth_mm: 10` in output JSON was geometrically impossible against an ~ 8 mm-tall shell (the `edge_dist` normalization used global max width instead of local per-slice width,

truncating the parabola early at the narrow heel). Fix verified: shell height now correctly reaches 13mm (3mm base + 10mm cup) when `heel_cup_depth_mm=10`.

- **Footprint shape — confirmed foot-like.** Width profile independently measured: ~47mm at heel, narrows to ~54mm at arch waist, widens to ~97mm at forefoot, tapers at toe. Watertight, 4,800 vertices / 9,596 faces (this exact vertex/face count appearing identically across all runs is NOT suspicious — it's a fixed function of mesh grid resolution `res_x=30, res_y=80`, independent of input parameters).
 - **`test_extreme_flags.py` flag-count bug — found and fixed.** The synthetic test's input dict omitted `cadence_steps_per_min`, which caused the cadence-check default `(metrics.get("cadence_steps_per_min", 0.0) == 0.0)` to spuriously fire, producing 3 flags instead of the expected 2. Fix (adding `"cadence_steps_per_min": 95.0` to the test's input dict) was independently verified: `1 passed`, exactly 2 correct flags.
 - **Foot-length-from-frontal-video — confirmed structurally limited, not a bug.** Added foot-length estimation from `LEFT_HEEL`/`LEFT_FOOT_INDEX` landmark distance. Result: ~34mm, not a real foot length. Root cause is real and architecturally important, not a coding error: frontal-view 2D pose foreshortens heel-to-toe distance because that axis runs toward/away from the camera, not across it. **No calibration hack should be applied to "fix" this** — it's a genuine limitation of single-camera frontal video for depth-axis measurements. The architectural conclusion: gait video should drive *functional* design parameters (rigidity, posting, arch support — everything measured across/within the camera plane, which frontal video handles well), while *sizing* (foot length/width) should come from a separate input (foot scan or manual entry), not be extracted from gait video.
-

6. Unresolved / open issues as of the last exchange — DO NOT treat these as settled

- **A contradiction is currently open and unresolved.** The most recent self-reported message claims (a) `rules.py` was refactored "two rounds ago" to return independent left/right parameter objects, and (b) `__main__.py` never wrote `params.json` to disk at all — the CLI only ever produced `.stl`, `.png`, and `print_instructions.json`. Both claims contradict directly-verified facts: the zip-based review (section 5) ran the actual `rules.py` and got a single flat `OrthoticParams` object, not a left/right split; and the project's own README (approved and re-uploaded unchanged across many rounds) explicitly documents a `python -m gaitform map-parameters ... -o data/params.json` CLI command. **This needs to be resolved by obtaining a fresh, complete zip of the entire current repository** (not a curated subset) and re-verifying `rules.py`'s actual return structure and whether `__main__.py` writes `params.json`, directly, before trusting any further claims about these two files.

- **Only 3 of ~10+ source files have ever been directly verified by execution:** `analyzer.py`, `rules.py`, `generator.py`. Never yet verified directly: `__main__.py` (CLI entrypoint), `gaitform/extraction/mediapipe_extractor.py` (the production pose extractor — a substitute was written for testing purposes, not their actual file), `gaitform/logging/db.py` (the SQLite logger — has the most history of contradictory/stale evidence), `gaitform/ui/app.py` (Streamlit app — every screenshot of it has come with an explanation involving caching issues, so there is no confidence the UI correctly calls the verified-correct backend modules end-to-end).
 - **The database logging path has unresolved history.** A forward-slash vs. backslash path-separator forensic difference was used to prove one specific DB row came from a manual insert script (`trigger_logging.py`) rather than the live Streamlit app, during a period when the Streamlit server was allegedly running stale cached code. This explanation is plausible and was eventually accepted as consistent, but the *underlying claim that Streamlit's live app correctly logs fresh, correct data when run normally* has still never been independently confirmed by direct execution.
 - **No real foot-scan/sizing input exists yet.** Shell dimensions are a fixed placeholder (260mm × 98mm) regardless of the actual person in the video. This is a known, accepted gap — not yet addressed, and was explicitly deferred as a separate future feature, not a bug to patch.
 - **The rules table has plausible-sounding inline rationale but no cited clinical sources.** The original spec called for "literature-backed rules" with documented provenance — current comments read as reasonable engineering judgment, not sourced clinical evidence. Needs real literature citations or clinical (podiatrist/orthotist) review before being treated as more than directionally sensible.
 - **Pronation angle is a kinematic proxy (knee-to-ankle vector angle from vertical), not a validated clinical pronation/eversion measurement.** Should be described precisely as such in any patent or clinical-facing language — don't conflate the two.
 - **No real validation against ground truth has been performed** for any absolute metric value (stride length, cadence, etc.) — only internal consistency/plausibility checks. Calibration currently assumes a fixed subject height (now a parameter with documented limitation, but no real height input or calibration reference exists yet).
 - `stance_time_pct`/`swing_time_pct` **remain hardcoded constants (60.0/40.0)** by design — never computed from real toe-off detection. This was explicitly kept out of scope in the most recent surgical fix round and remains a known, accepted simplification.
-

Does achieve: a real, independently-verified, working demonstration of the core inventive bridge — dynamic gait video genuinely driving changes in orthotic design parameters and 3D geometry, with full input-to-output traceability. This is sufficient evidence for patent reduction-to-practice purposes (not a legal opinion — confirm with a patent attorney).

Does not yet achieve: anything usable on an actual patient. No real foot sizing, no clinically validated metrics, no cited clinical rules, no real outcome data in the dataset yet. The path from here to patient-usable requires: real foot-scan/sizing input, metric validation against a reference system, clinical/literature sourcing for the rules table, and a real pilot to populate `clinician_outcome_notes`. None of this blocks patent conversations; all of it is required before clinical use.

8. Working method to carry forward

1. Never accept a "this is fixed" or "here is the proof" claim without independently running the actual code, when source files are available. Pasted terminal output, screenshots, and prose descriptions have repeatedly not matched reality in this project.
2. When given a zip or files, build a minimal harness if needed (e.g., a compatible `KeypointSequence`/extraction substitute) to actually execute the real modules rather than reading them and trusting behavior by inspection alone.
3. Watch for telltale signs of stale or fabricated data: values that are bit-identical across supposedly-independent runs while sibling values differ; file path separator conventions (Windows backslash vs. forward slash) that reveal which script actually produced a given artifact; timing values in test output that don't plausibly change between runs.
4. When issuing fix instructions, scope them surgically — name the exact file(s), the exact change, explicit "do not touch" lists, and a verification step that requires pasting raw, complete output. This project's history shows that vague fix requests reliably result in scope creep and unverifiable claims.
5. Keep the medical-device/non-diagnostic framing intact in all README, UI, and code-comment language, regardless of how good a demo result looks.